

伝わる発信のつくりかた

@yoshiko

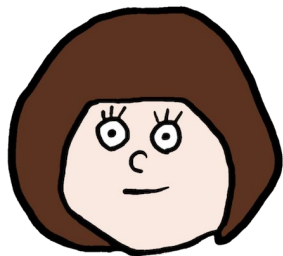
2026/06/19 Zennfes



登壇や記事など、自分の発信について
わかりやすいと言っていたることが多いので
発信がうまくまとまらず悩んでいる方に向けて、
自分が何かを発信する際の思考の仕方、
作るときに気をつけていることをシェアします。

正解はないので、あくまで私のやりかたということで参考になれば

自己紹介



よしこ

株式会社ナレッジワーク 専門役員 Principal
Google Developer Experts for Web

— X (Twitter)
@yoshiko_pg

— Zenn
@yoshiko

— Web
<https://yoshiko-pg.github.io>



<https://github.com/yoshiko-pg/difit>

diffビューアのOSSつくっています

目次

- ネタを考える
- 構成を考える
- 紙面を考える
 - スライドの場合
 - 記事の場合

目次

- **ネタを考える**
- 構成を考える
- 紙面を考える
 - スライドの場合
 - 記事の場合

コアメッセージ

ターゲット

まずこのふたつをはっきりさせる

コアメッセージから考える場合



コアメッセージ

ターゲット

コアメッセージから考える場合

コアメッセージから考える場合

「発信を見た人に持ち帰ってもらいたいことを、ひとことというとな何？」

コアメッセージ



SPAのStateは
「サーバーデータのキャッシュ」
「Global State」「Local State」
の3種類に分類できる



技術面の健全化であっても
課題ドリブンで進める

コアメッセージからターゲットを連想する

←
「発信を見た人に持ち帰ってもらいたいことを、ひとことでいうと何？」

ターゲット

コアメッセージ



ReduxのようなSingle Stateは運用したことあるが
SWRのような最近のState管理のトレンドを知りたい人
→ Reactの開発経験はあるはず。
具体コードを出しても伝わりそう



機能開発と負債返済の両立に苦しんでいる人
少人数チームをリードする裁量と責任がある人
→ 具体的な技術の話よりも抽象度を上げて
プロジェクトマネジメント視点の事例が参考になりそう

コアメッセージからターゲットを連想する

コアメッセージがあれば発信はできるが、そこから連想したターゲットを脳内に据えることでコアメッセージ自体は変わらなくても**伝え方をチューニングできて、伝わりやすくなる。**

- 今回伝えるコアメッセージが一番刺さりそうな人はどんな人？
 - ジュニア？ ミドル？ ベテラン？
 - 特定の職種やレイヤー？
 - 特定の技術利用者？

例：この技術解説は初心者の人にも読んでほしいから、この用語には補足を入れておこう

例：社内メンバー向けだし実感を持ってほしいから、架空の例じゃなくて社内の実例を入れよう

ターゲットから考える場合



コアメッセージ

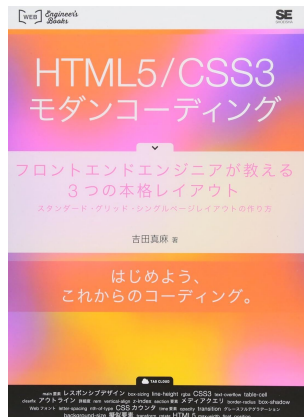
ターゲット

ターゲットから考える場合

ターゲットから考える場合

「今回発信する内容はどんな人に向けたもの？」

ターゲット



HTML/CSS初学者は脱してるけど、
準初級者から中級者へステップアップしたい人
カッコいいサイトを作りたい人



最近Vibe Codingってよく聞くから気になるけど
どういうものなのかわからなくて手がでない人

ターゲットからコアメッセージを連想する

「今回発信する内容はどんな人に向けたもの？」

コアメッセージ

ターゲット



文法を学ぶのではなく実践的なハンズオン
各プロパティを深ぼれる内容にする
サンプルサイトのデザインにもこだわる



「私はこのツールでこんなふうやってます」
思ったより簡単、自分もやれそうと思ってほしい

ターゲットからコアメッセージを連想する

対象や状況から内容を思いつかせる方法。

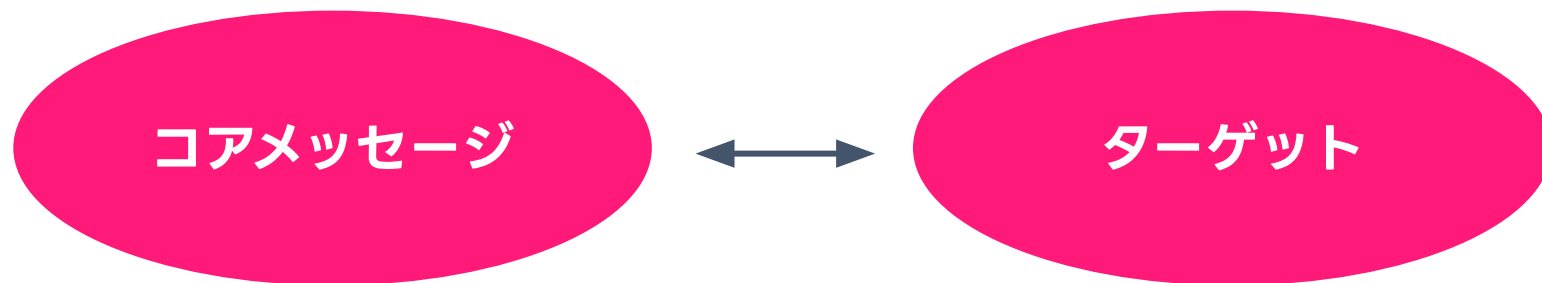
- ・ イベント登壇
- ・ メディアへの寄稿
- ・ 書籍の執筆

これらのように、**自分以外が企画する場合**へ**自分が発表を持ち込む**場合、場のターゲットから考えやすい

- ・ その場はどういった層をターゲットにしているのか？ 初心者？ 自分と同じぐらい？
- ・ イベントや企画のタイトルは？ コンセプトは？ 参加者は何を期待して集まっている？
- ・ その人達に一番響きそうな難易度・抽象度の話ってなんだろう？

今日はZennfes = 記事や発信に興味のある方が多いかな？ と思い、発信についての登壇にしました！

完全にターゲットからだけネタを逆算するというより、**自分の中の「これ話せそう」を固める手がかりに**



実際は一方通行ではなく、相互にぐるぐる深めています

例外：発信に別の目的がある場合

発信の目的が「**誰かに何かを伝える**」であれば、前述のように**ターゲット**と**コアメッセージ**でいい。
でもそれ以外の目的があるなら、**その目的を達成できるような角度**から考える必要がある。

実例：事業がステルスなせいで採用マーケティングできてないから、技術面から会社の知名度あげたい！

→ [とにかく網羅的に技術アピールできるような構成をとった連載記事](#)

例題：自分が好きで詳しい技術があるので

「この技術に詳しい人」というブランディングをしてみたい！

例題：PVが収入になる寄稿で、依頼元からも高いView数を期待されている！



目次

- ネタを考える
- **構成を考える**
- 紙面を考える
 - スライドの場合
 - 記事の場合

話したいことを洗い出す

最初からスラスラ書けるわけではないので、ワードクラウドみたいに思いついたことをばーっと単語でもいいからメモっていく。あとから組み立てる。

メモ

- やったこと
 - 課題の洗い出し
 - 優先度決め
 - ロードマップ作成
 - 実行

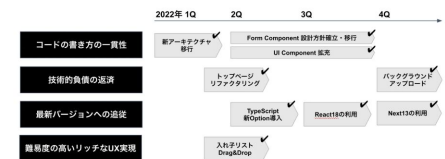
実例

- ロードマップ
- 2022

1ZITmR3yu4_HaD93B7r7udYJlw64G7eZABN8RHQ7zcO4

実現タイムライン

_KNOWLEDGE WORK

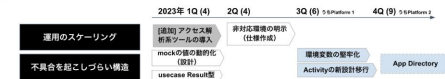


2023

1rNA_Ka54gux3S8AlvNoPc-APR052UByl6h7amn-zOj4
<https://docs.google.com> Content not found

実現タイムライン (2Q再考-1)

_KNOWLEDGE WORK



チームの自律性と共通部品の最大化、このふたつを極めないとこのスピードと数を高品質で実現できない

ひとつのnext.jsアプリケーションを複数のnext.jsアプリケーションに割る

垂直マイクロフロントエンド

そのために大部分をnpm packageに切り出す

pnpm workspace

turborepo

E2E

Vercelの記事にもあったよ

抽象

コンパウンド

3年で10こ

技術

コンパウンドには各チームの自律と共通部品が必要

共通部品→次のセッションで！

各チームを自律させるためのデプロイの分離。そのためのアプリケーション分離 Bambooの紹介

フロントエンドでは？

ひとつのnext.jsアプリケーションを複数のnext.jsアプリケーションに割る

垂直マイクロフロントエンド

そのために大部分をnpm packageに切り出す

pnpm workspace

turborepo

E2E

情報伝達はアウトラインが命

今日はこれだけ覚えて帰ってください
(つまりこれが今日のコアメッセージです)

アウトラインの実例



今日話す内容

ネタを考える

コアメッセージ

コアメッセージの実例

ターゲットを連想する

ターゲット

ターゲットの実例

コアメッセージを連想する

例外

構成を考える

話したいことを洗い出す

アウトラインを考える

⋮

前提

リアーキテクチャの変遷

最低限期

モノリスフロントエンド期

プロダクト分割期

進め方

課題の洗い出し

優先度決め

ロードマップ作成

実行

むすび

全体像

1. サーバーデータのキャッシュ

SWRを使ったキャッシュ

2. Global State

Recoilを使ったState管理

3. Local State

要件定義

技術選定

コードベース作成

微調整

★ 同じ色の見出しは同じ水準、を意識！

★ 見出しの並びだけでも発信内容が伝わるか？

👉

よく使う構成

説明系

- 今日話すこと
- 前提
- トピック 1
 - 説明
 - 事例 or 具体例
- トピック 2
 - 説明
 - 事例 or 具体例
- まとめ

課題解決系

- 今日話すこと
- 前提 ★手法に関わるので重要
- 課題
- 解決手法
 - 説明
 - なぜそれを選んだか
 - 事例 or 具体例
- 結果
 - 解決されたか
 - よかったこと
 - 悪かったこと
- まとめ

実践共有系

- 何について共有するか
- ステップ 1
- ステップ 2
- ステップ 3
- アウトプット・結果

リハーサルで調整

元々

5分

<戦略> 事業戦略から技術戦略へ

事業戦略を理解する

技術戦略を立てる

5分

<技術> 具体的な移行の話

構成のBefore/After

実現タイムライン

使った主な技術

⋮

抽象と具体を半々で話す構成

リハでの同僚Feedback

- ・ 抽象の話が長い。
技術的な結論を先に知りたい
- ・ 技術のイベントなので、
技術部分を厚めに聞きたい
- ・ 技術的にぶつかった
課題と解決策が聞きたい

調整後

1分

技術の結論サマリ

3分

<戦略> 事業戦略から技術戦略へ

事業戦略を理解する

技術戦略を立てる

6分

<技術> 具体的な移行の話

構成のBefore/After

実現タイムライン

使った主な技術

ぶつかった課題

⋮

冒頭に簡単に技術の掴みを入れ、
抽象部分を薄く技術部分を濃く

構造が明瞭だと時間の再配分も楽！

登壇はリハーサル大事！ 一度口に出して喋ってみるのがおすすめ

 リハーサルで時間を計りましょう！ 

アウトラインのセクションごとに所要時間目安があるといい

同僚や友達などリハを聞いてくれる相手がいるとモチベUP

目次

- ネタを考える
- 構成を考える
- **紙面を考える**
 - **スライドの場合**
 - 記事の場合

アウトライン通りにページを作るだけ！

なので、簡単にTipsを紹介します

目次をセクション代わりに使う

今日話す内容

ネタを考える

コアメッセージ

| コアメッセージの実例

| ターゲットを連想する

ターゲット

| ターゲットの実例

| コアメッセージを連想する

例外

構成を考える

| 話したいことを洗い出す

| アウトラインを考える

⋮

目次

- ・ ネタを考える
- ・ 構成を考える
- ・ 紙面を考える
 - ・ スライドの場合
 - ・ 記事の場合

6

目次

- ・ ネタを考える
- ・ 構成を考える
- ・ 紙面を考える
 - ・ スライドの場合
 - ・ 記事の場合

19

話したいことを洗い出す

アウトラインを考える

リハーサルで調整

各スライドに見出しをちゃんと入れる
見出しだけを読み流しても
内容の流れがわかるように

目次を中扉として使うと、
今何を話しているのかわかりやすい

下の方に重要な文字を入れない

今回そこまで守れなかったかも。。
念頭にはおいてます

会場では、前の人の頭でスライドの下の方が見えないことがよくある
下部はページ番号や補足など、見えなくても困らないものに

重要な情報は、上半分～真ん中に置く

このへんの文字は、後ろの方では頭で見えない…

基本的なところですが、誰でも読みやすい表示を心がけるのはWebと同じ

フォントサイズ

✗ 小さい文字は後ろの席から読みにくい…

○ 大きい文字は読みやすい！

コントラスト

✗ 背景に近い色は読みにくい

○ 濃い色の文字は読みやすい！

「スライド10枚/発表20分/フォントサイズ30pt」ルールや

「聴衆の最年長者の人の年齢÷2 = 最小フォントサイズ」

なんて提言もあるぐらい

ref. <https://www.gsb.stanford.edu/insights/10-steps-perfect-your-startup-pitch>

コントラストの基準参考： Web Content Accessibility Guidelines

大きい文字であれば少し基準が下がる。

薄めの色をどうしても使いたいのであれば大きく太くすると良さそう

ref. <https://waic.jp/translations/WCAG21/#contrast-minimum>

AI文体を消す

ドラフトにAIを使っても、最後は自分の言葉でリライトすると自分の声色で真っ直ぐに伝わります。
AI感丸出しだと、内容がよくても文体がノイズになってしまいもったいない…

登壇資料の「構成・言葉・余白」

— 何を話し、何を削るのか —

発信は前に立って話すことではない、構造化と伝達だ。

「聞き手を迷子にしない」デザインが効く。

“ちゃんと伝わる” かどうかは、書く前に決まっている

伝わる発信のつくりかた

コアメッセージをはっきりさせる

情報伝達はアウトラインが命！

登壇はリハーサル大事！

tropes.md や [stop-slop](https://stop-slop.com) など、AI文体のサンプル集があるので日本語に翻訳して掴んでみましょう
(一度気付いてしまうようになるともう戻れない)

書いてある文章をそのまま読み上げない

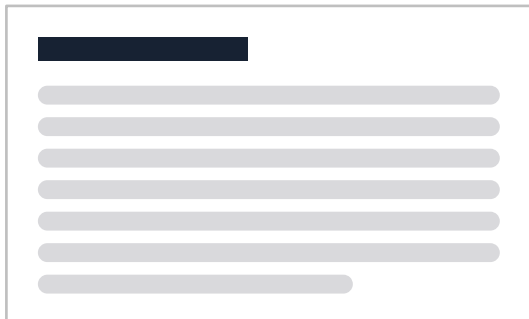
今回そこまで守れなかったかも。
念頭にはおいてます

書いた文章をそのまま読み上げていると、聞いてもらえず文字を読まれてしまう。

とはいえ、あとから資料だけで共有したときにわかる程度の文章は入れておきたい。

それなりに文章は入れつつ、話すときは表現崩したり、口頭だけの内容を入れたりしています

✗ 聞いてもらえず読まれてしまう



○ コアな内容のみ



✗ あとから資料だけ見て伝わらない



サンプルコード

コードは相当簡単な内容でないと、初見ではほぼ理解できない。

4行ぐらいのボリューム + 丁寧な解説、でないと表示時間内にわからないなという所感。

シンタックスハイライトは必ずいれておきたい。**SlidesCodeHighlighter** というツールが便利！

<https://romannurik.github.io/SlidesCodeHighlighter/>

テーマやフォント選べるし、コードをURL末尾クエリに保存できるので、あとから編集も楽々

```
const fruits = ['apple', 'banana']
const upper = fruits.map(
  (f) => f.toUpperCase()
)
```

```
const fruits = ['apple', 'banana']
const upper = fruits.map(
  (f) => f.toUpperCase()
)
```

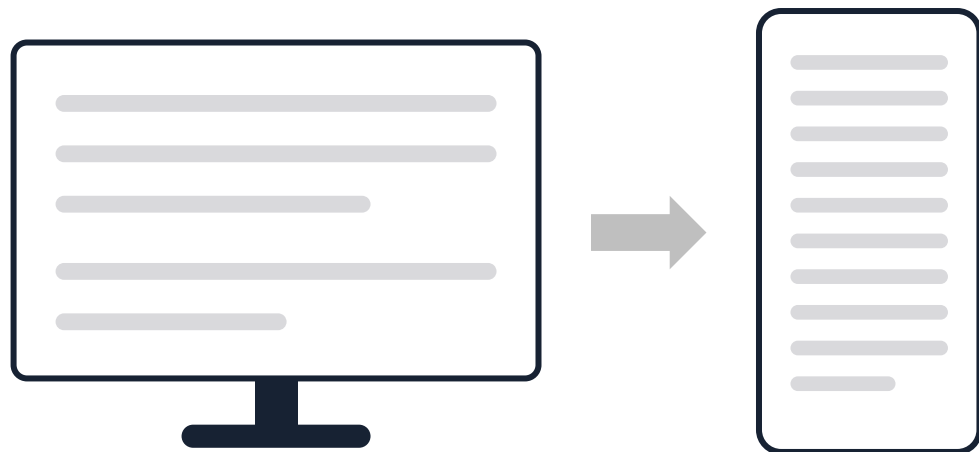
目次

- ネタを考える
- 構成を考える
- **紙面を考える**
 - スライドの場合
 - **記事の場合**

アウトライン通りにセクションを区切って書くだけ！

なので、簡単にTipsを紹介します

PCとスマホ両方で見ると



同じ文章でも、スマホでは段落が長くなる
改行位置も変わる

PCで書き終わった記事をスマホで見ると、
かなり印象が変わります。

紙で印刷して読むと誤植に気付ける、の
感覚に近いかも

違和感の出た表現をちょっと変えたり、
段落を区切ったり・まとめたり調整。

記事全体がなるべく短くまとまった印象にな
るようにしています

記法を適切に使う

特にZennは独自記法がいくつかあるので、適切なものを選んで見た目もセマンティクスも向上！
公式の「ZennのMarkdown記法一覧」を一度眺めておくのがおすすめ

`:::message`



補足やお知らせなどに便利

`:::message alert`



重要な注意・警告に使える

`:::details` タイトル

▶ タイトル

▼ タイトル

表示したい内容

````js:foobar.js`

fooBar.js

```
const great = () => {
 console.log("Awesome")
}
```

## AI文体を消す

---

記事でも同じ！

自分の声として発信できるようにバランスをとりつつも、  
AIのサポートをうまく取り入れる方法を過去に記事書いてるのでよければ参考にしてみてください。

[個人的 AI Writing のやりかた](#)

おわりに

---

いったん以上

なんか、発信のハードル上がったな… 🙄

おわりに

---

大丈夫 😊

**みんなそんな真剣に見てないので 😊**



**みんなそんな真剣に見てないので 😊**

みんな気楽に聞いているので、気楽に話せばいい  
話してる側がガチガチだと聞く方も疲れるし

って自分に言い聞かせてます

**準備は真摯に、発信は気楽に**

# おわり

ありがとうございました

2026/06/19 Zennfes

